You use `d3.svg.line` when you want to draw most shapes and lines, whether filled or unfilled, or closed or open. You should use `d3.svg.area()` when you want to draw a shape where the bottom of the shape can be calculated based on the top of the shape as you're drawing it. It's suitable for drawing bands of data, such as that found in a stacked area chart or streamgraph.

By defining the `y0` function of `d3.svg.area`, we've mirrored the path created and filled it as shown in figure 4.23, which is a step in the right direction. Notice that we're presenting inaccurate data now, because the area of the path is twice the area of the data. We want our areas to draw one on top of the other, so we need `.y0()` to point to a complex stacking function that makes the bottom of an area equal to the top of the previously drawn area. D3 comes with a stacking function, `.stack()`, which we'll look at later, but for the purpose of our example, we'll write our own.

Listing 4.14 Callback function for drawing stacked areas

```
var fillScale = d3.scale.linear()
            .domain([0,5])
            .range(["lightgray","black"]);
var n = 0;
for (x in data[0]) {
    if (x != "day") {
        var movieArea = d3.svg.area()
            .x(function(d) {
                return xScale(d.day)
            })
            .y(function(d) {
                return yScale(simpleStacking(d,x))
            })
            .y0(function(d) {
                return yScale(simpleStacking(d,x) - d[x]);
            })
            .interpolate("basis")

        d3.select("svg")
            .append("path")
            .style("id", x + "Area")
            .attr("d", movieArea(data))
            .attr("fill", fillScale(n))
            .attr("stroke", "none")
            .attr("stroke-width", 2)
            .style("opacity", .5);
        n++;
    };
};
function simpleStacking( incomingData, incomingAttribute) {
      var newHeight = 0;
      for (x in incomingData) {
        if (x != "day") {
          newHeight += parseInt(incomingData[x]);
          if (x == incomingAttribute) {
```

Creates a color ramp that corresponds to the six different movies

We won't draw a line for the day value of each object, because this is what provides us with our x coordinate.

Each movie corresponds to one iteration through the for loop, so we'll increment *n* to use in the color ramp. We could also create an ordinal scale assigning a color for each movie.

A d3.svg.area() generator for each iteration through the object that corresponds to one of our movies using the day value for the x coordinate, but iterating through the values for each movie for the y coordinates

Draws a path using the current constructor. We'll have one for each attribute not named "day". Give it a unique ID based on which attribute we're drawing an area for. Fill the area with a color based on the color ramp we built.

Finishes the for loop, increments to the next attribute in the object, and increments *n* to color the next area

This function takes the incoming bound data and the name of the attribute and loops through the incoming data, adding each value until it reaches the current named attribute. As a result, it returns the total value for every movie during this day up to the movie we've sent.